

TABLE-SMITH: Structured Data Transformation Through Class-Based Segmentation

M. Deepika
CSE (Artificial Intelligence & Machine Learning)
Neil Gogte Institute of Technology,
Hyderabad, India
deepikamoksh2004@gmail.com

Shilpa Choudhary
CSE (Artificial Intelligence & Machine Learning)
Neil Gogte Institute of Technology,
Hyderabad, India
shilpachoudhary2020@gmail.com

B Sai Charan
Student, Computer Science and Engineering
Neil Gogte Institute of Technology,
Hyderabad, India
saicharanbachu421@gmail.com

K Hari
Student, Computer Science and Engineering
Neil Gogte Institute of Technology,
Hyderabad, India
harikodakandlaa@gmail.com

K Anirudh Sai
Student, Computer Science and Engineering
Neil Gogte Institute of Technology,
Hyderabad, India
anirudhsaikommuri@gmail.com

Sandeep Kumar
Department of Computer Science & Engineering
Symbiosis Institute of Technology,
Nagpur Campus
Nagpur, India
er.sandeepsahratia@gmail.com

Abstract—In the current era of rapid technological advancements, data plays a significant role in almost every task. Owing to the integration of technology among various industries, there has been a rapid influx of data. This has raised the need to immediately develop an efficient system capable of organizing and transforming inconsistent data. Poorly formatted tables and missing records often disturb smooth data exchange and integration. Table-Smith implements a smart approach to analyze, restructure, and consolidate tabular datasets that originate from several sources. By aligning positional feature understanding with reasoning, the system renders clean and noisy tables to produce standard machine-readable output. Powered by Python, Flask, and transformer-based large language models (LLMs), Table-Smith accelerates tasks such as data normalization, type alignment, and schema synchronization. Rigorous evaluation on multi-domain datasets illustrates that the framework accomplishes a higher accuracy in contrast with existing solutions, while offering explainable and easily deployable results. The implementation is available on GitHub. <https://github.com/psgroup433/Table-Smith>

Index Terms—Data Processing, Data cleaning and Transformation, Large Language Models (LLMs)

I. INTRODUCTION

In the present digital era, the performance of an organization greatly relies on its capability to manage and utilize data collected from different sources, internal or external. These datasets often vary in structure, format, and meaning, making their merging a difficult task. Even simple elements such as academic or financial records can appear under different names or styles in different systems. Such differences make it difficult to merge data accurately and can hinder analytical processes as well as intelligent systems that depend on well-structured tables for tasks such as prediction, recommendation, or information retrieval [1][2].

While there are numerous works on automating table transformation and schema correspondence, existing systems mainly concentrate solely on partial transformation types or

do not provide a clear explanation why a certain action has been applied. This is particularly challenging in cases of data semantics (e.g., codebook categories must be translated into descriptive text, or numeric values must be translated between measurement systems) [3][5].

To overcome these challenges, Table-Smith presents a consolidated interface that streamlines table cleaning and data integration across various file formats such as CSV, JSON, and PDF. As most of the prior works are based on a database or a single file system and use SQL queries to perform the operations, Table-Smith focuses on a more fundamental problem created due to human intervention: the problem of multiple formats and multiple data storing methods. The system begins with a pre-processing phase that removes noise and duplicate entries, leading the data into consistent formats. Following this, a classification-transformation pipeline organizes and converts the input tables.

By combining feature extraction, semantic alignment, and JSONL-ready output generation, the framework not only enables instant analytical use but also supports the fine-tuning of large language models for structured data reasoning.

The aim of this study is to achieve two objectives:

- Learning sensitive and robust transformations of structural and conceptual shifts in tabular data.
- Deriving transformation logic so that it can be understood, reused, and validated when the transformation is pushed into actual environments.

Taking this approach, Table-Smith improves the consistency and understanding of data integration flows and provides an adaptable method for processing large-scale tables across financial institutions, healthcare organizations, and educational institutions.

II. BACKGROUND AND MOTIVATION

The present era has seen an exponential growth in data collected and information generated. Particularly there has been a rapid growth in tabular information. While the goal of automating data transformation has existed for decades, it still needs to evolve to fit the ever-changing needs of developers. The techniques and tools used to achieve automation are in the process of evolving with advances in computing and artificial intelligence.

Even prior to the invention of LLMs, systems like DataX-Former proposed to automate transformations by mining web tables and forms [5][7]. While efficient, these methods lacked the deep semantic reasoning of modern neural models. Even the leading corporations and major players of the ETL industry are also focusing on developing and integrating machine learning algorithms and artificial intelligence in their software. The development of foundational models pre-trained for different data transformation is shown in Table I.

Recent advancements in the field include:

Md.Mahadi Hasan Nahid and Davood Rafiei [8]: LLMs such as ChatGPT are excellent at capturing and generating text. They are also capable of processing tables, although they tend to run into trouble with tables from the web due to messy formats and dissimilar values. To address this, they developed NormTab, designed to clean and organize web tables utilized by LLMs.

Tianshu Zhang et al. [9]: Proposed a new dataset with many genres of real-world tables. They fine-tuned the Llama language model using a method called LongLoRA, aiming to create a “generalist” model for tasks like entity linking and column annotation.

Arash Dargahi Nobari and Davood Rafiei [10]: Proposed the Deep Tabular Transformer (DTT), which automatically transforms data between structural formats. DTT uses language model-driven learning to infer transformation logics from just a few aligned source–target row pairs. In subsequent work [11], they proposed a more efficient algorithm to find an optimal transformation path for accurate table joinability.

Based upon these works, Nobari and Rafiei [12] proposed TabulaX, a comprehensive framework that leverages large language models to address various categories of table transformation problems.

III. PROPOSED WORK

Table-Smith is a conceptualized multi-stage modular framework developed for transforming, formatting, and interweaving heterogeneous tabular datasets that are in essence same or similar. It is designed according to the divide and conquer principle, allowing complex tasks to be broken down. The process begins with the upload of two or more files containing tabular data which are then sequentially processed through our custom designed pipeline (Fig. 1).

Unlike traditional ETL tools that operate on pre-defined workflows, this architecture is dynamic. Table-Smith introduces a dynamic mechanism that combines classification-based pre-processing with LLM-driven transformation applied

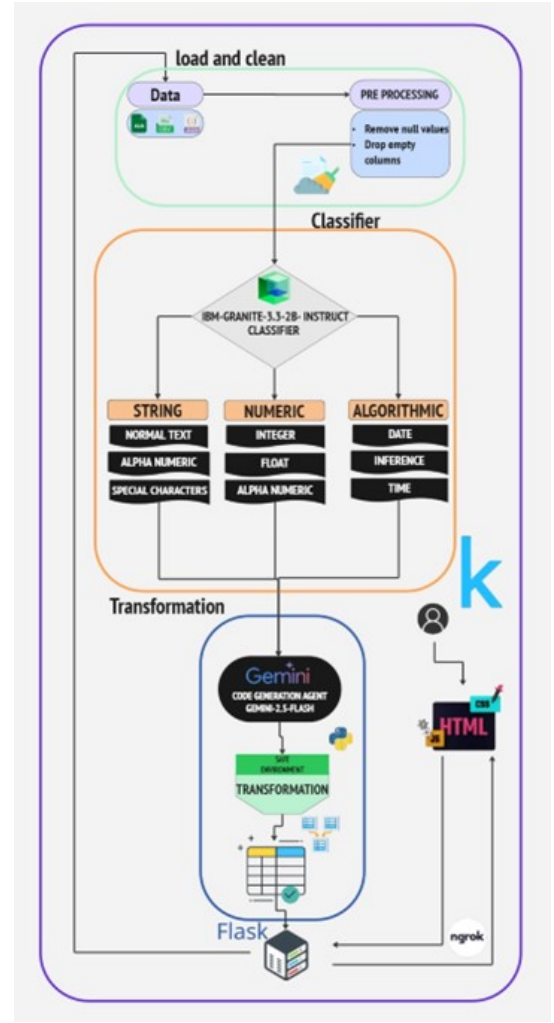


Figure 1. Architecture of Proposed Work. **Table-Smith** combines classification-based pre-processing with LLM-driven transformation.

via code generation. This approach allows the framework to infer the semantic relationship between columns automatically. This generated code when executed produces structured, machine-readable and human understandable outputs.

A. Dataset and Pre-processing

To evaluate the efficiency and adaptability of the system, a custom dataset comprising 540 tables containing approximately 8000 columns was compiled. The dataset has synthetically added noise and consists of files in both CSV and JSON formats. A series of pre-processing operations were executed:

- 1) Elimination of null or unnamed columns.
- 2) Removal of duplicate entries.
- 3) Standardization of column names into the snake_case convention.
- 4) Coalescing columns using Granite’s analysis.

Table I
COMPARISON OF DATA TRANSFORMATION SYSTEMS

Feature	TabulaX	DATAMORPHER	DataXFormer	Table-Smith
Core Methodology	Classify-then-Transform	Direct Code Generation	Example-based Discovery	Analyse-then-Generate
Primary Knowledge Source	LLM Parametric	LLM Parametric	Web Tables & Forms	LLM Parametric
Transformation Mechanism	Mixed (Code-gen, Curve Fit)	Python/SQL Code Gen	Lookup Table / Formula	Python Code Generation
Classification Granularity	4 High-Level Classes	N/A (Zero-Shot)	N/A	AI-driven analysis
Interpretability	Moderate (Class + Function)	Moderate (Code only)	High (Source table shown)	High (Analysis + Code)

B. Table-Smith Multi-Stage LLM-Driven Transformation (TS-MLT)

Input: Two or more heterogeneous tables $T_1, T_2, \dots, T_n$ in CSV/JSON/XLSX formats.

Output: Unified structured table T^* with semantic consistency and transformation notes.

1) **Step 1 — Format-Adaptive Ingestion:** Detect file format using and file header parsing. Convert all into intermediate pandas DataFrame objects. Normalize headers:

$$\forall c \in \text{DataFrame}, c \leftarrow \text{to_snake_case}()$$

2) **Step 2 — Class-Based Semantic Classification:** Sample rows are fed into Granite-3.3-2B-Instruct (LLM_1). LLM_1 returns a tuple (class, subtype, cleaning_note) for each column, where $\text{Class} \in \{\text{String}, \text{Numeric}, \text{Algorithmic}\}$. Generate a structured JSON schema:

```
{ "column": "Satisfied ", "class":
"Numeric", "subtype": "percentage_float",
"note": "Convert percentage strings
('47.6%', '100%') to floats (0.476, 1.00)
and remove percentage signs and trailing
zeros." }
```

3) **Step 3 — Context-Aware Rule Mining:** For each pair of candidate columns across tables, compute semantic similarity using embeddings:

$$\text{sim} = \text{cosine}(\text{Embed}(c_i), \text{Embed}(c_j)) \quad (1)$$

Where If $\text{sim} \geq \theta$, mark as potentially joinable. Use self-refinement via LLM_1 to confirm relationships.

4) **Step 4 — Dynamic Code Synthesis:** Pass classification + schema + cleaning notes to Gemini-2.5-Flash (LLM_2). The prompt requests Python transformation code to merge columns with specific logic. The system executes the returned code and validates output shape. On failure, it appends the trace-back and re-prompts LLM_2 (self-healing retry loop).

5) **Step 5 — Cross-Validation and Scoring:** Compare transformed output with synthetic “gold” rules or checksum logic. Compute accuracy metrics per class.

$$\text{acc} = \frac{\text{correct_predictions}}{\text{total_predictions}} \quad (2)$$

IV. RESULTS AND DISCUSSIONS

A. Experimental Setup

Table-Smith was deployed in a cloud-based Kaggle Notebook environment using Python-3 and an NVIDIA Tesla T4

GPU. The main LLMs used were Granite 3.3-2B-Instruct (via Hugging Face) and Gemini/2.5-Flash (via Google API). The LLMs were integrated with few pre-processing libraries such as (pandas, NumPy, re) and Flask-based APIs to control data pipelines.

B. Quantitative Analysis

The model showed an overall accuracy of 0.84–0.86 across all classes, with high performance in conceptual classifications of columns compared with other LLMs (such as LLaMA, Mistral, Phi, and Qwen).

Table II
LLM ACCURACY GROUPED BY PRIMARY DATA TYPE

Row Labels	mistral	llama	phi	qwen	granite
Date	0.855	0.840	0.851	0.911	0.912
Numeric	0.658	0.758	0.795	0.814	0.802
String	0.744	0.833	0.814	0.847	0.865
Mean	0.737	0.806	0.815	0.849	0.851

As depicted in Table II, Granite reached a mean accuracy of 0.851, crossing Qwen (0.849) by just 0.2% and ahead of LLaMA, Phi, and Mistral. Within the Date category, Granite attained 0.912, which is notably higher than most other models.

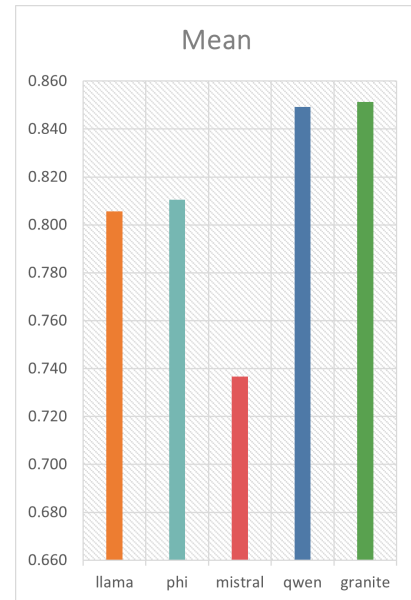


Figure 2. Mean Accuracy graph of different LLMs

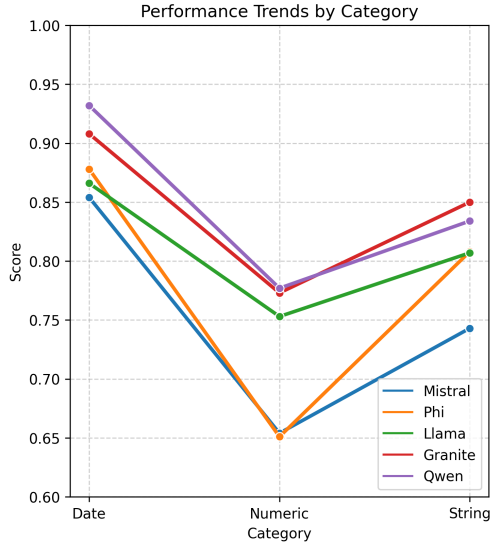


Figure 3. Various models' on primary class types

Table III
ACCURACY OF LLMs ACROSS VARIOUS DATA SUB-CLASSES

Sub-Class	mistral	llama	phi	qwen	granite
boolean	0.727	0.886	0.818	0.750	0.923
category_text	0.849	0.947	0.825	0.950	0.929
count_integer	0.375	0.490	0.677	0.552	0.596
date_iso_date_only	0.971	1.000	0.993	1.000	0.929
date_iso_date_timestamp	0.800	0.800	0.820	0.730	0.960
date_value	0.800	0.800	0.745	0.959	0.993
datetime_with_offset	0.891	0.700	0.782	1.000	0.809
free_text_notes	0.800	0.812	0.824	0.841	0.836
general_string	0.788	0.804	0.916	0.872	0.884
integer_id	0.494	0.705	0.868	0.853	0.630
mixed_numeric_text	0.632	0.603	0.743	0.639	0.718
mixed_string_date	0.759	0.810	0.784	0.778	0.778
mixed_string_float	0.600	0.564	0.707	0.600	0.743
mixed_string_integer	0.625	0.588	0.723	0.656	0.728
mixed_type_complex	0.600	0.600	0.600	0.600	0.600
monetary_float	0.308	0.962	1.000	1.000	1.000
monetary_integer	0.625	0.500	0.625	0.750	0.625
numeric_float	0.933	0.993	0.943	0.951	0.983
numeric_integer	0.757	0.892	0.500	0.780	0.767
percentage_float	1.000	1.000	0.933	1.000	1.000
person_name	0.951	1.000	0.839	1.000	0.953
string	0.780	0.800	0.820	0.910	0.930
string_id	0.514	0.882	0.750	0.978	0.878
time_hh_mm_ss_text	0.907	0.927	0.983	0.998	1.000
url_string	0.800	1.000	1.000	0.933	1.000
year_integer	0.867	0.881	0.976	1.000	0.943
Mean	0.737	0.806	0.815	0.849	0.851

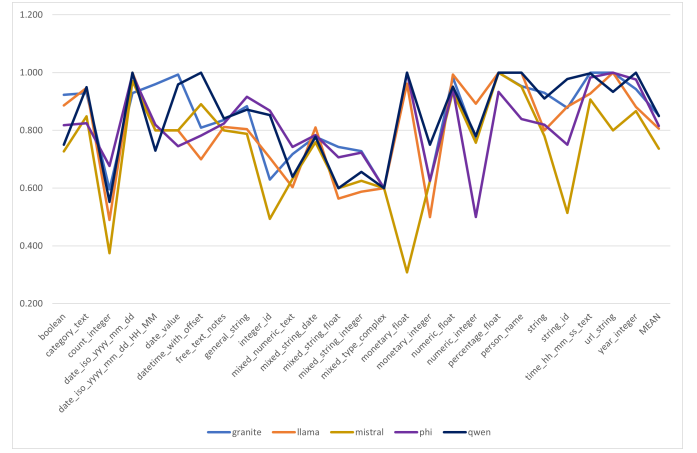


Figure 4. Sub-Class Wise Performance of different LLMs.

V. CONCLUSION

Table-Smith thus exemplifies a general, scalable approach to data transformation: we have shown that it is possible for basic LLM architectures to be adapted for automatic cleaning of a diverse range of tabular datasets with little need for fine-tuning on domain-specific tasks. As Granite-3.3-2B-Instruct for semantic classification and Gemini 2. 5-Flash for dynamic code synthesis, the benchmark achieves an average accuracy of 0.851. This architecture outperforms comparable models like LLaMA (0.806) and Mistral (0.737), particularly in interpreting temporal data where it reaches 0.912 accuracy. Rather than acting solely as a static ETL tool, Table-Smith functions as an intelligent, self-healing pre-processor capable of converting messy, diverse inputs (CSV, JSON) into standard, machine-readable formats, ready to be used as datasets. Future Enhancements Include: Expansion to Unstructured (Non-Tabular) Data, Multi-Table Constellation Joins, Specialized Fine-Tuning for Hybrid Data Types, Retrieval-Augmented Generation (RAG) for Contextual Disambiguation.

REFERENCES

- [1] R. J. Miller, F. Nargesian, E. Zhu, C. Christodoulakis, K. Q. Pu, and P. Andritsos, "Making open data transparent: Data discovery on open data," *IEEE Data Eng. Bull.*, vol. 41, no. 2, pp. 59–70, 2018.
- [2] A. Dargahi Nobari and D. Rafiei, "Dtt: An example-driven tabular transformer for joinability by leveraging large language models," *Proc. ACM Manag. Data (SIGMOD)*, vol. 2, no. 1, Mar. 2024.
- [3] S. Omidvartehrani, A. D. Nobari, and D. Rafiei, "Gxjoin: Generalized cell transformations for explainable joinability," in *Advances in Databases and Information Systems*. Cham: Springer Nature Switzerland, 2024, pp. 123–137.
- [4] Y. He, X. Chu, K. Ganjam, Y. Zheng, V. Narasayya, and S. Chaudhuri, "Transform-data-by-example (tde): an extensible search engine for data transformations," *Proceedings of the VLDB Endowment*, vol. 11, no. 10, pp. 1165–1177, 2018.
- [5] Z. Abedjan, J. Morcos, I. F. Ilyas, M. Ouzzani, P. Papotti, and M. Stonebraker, "Dataxformer: A robust transformation discovery system," in *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, 2016, pp. 1134–1145.
- [6] Y. He, K. Ganjam, and X. Chu, "Sema-join: Joining semantically-related tables using big table corpora," *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1358–1369, Aug. 2015.

- [7] Y. Wang and Y. He, “Synthesizing mapping relationships using table corpus,” in *Proceedings of the 2017 ACM International Conference on Management of Data*, ser. SIGMOD ’17. New York, NY, USA: Association for Computing Machinery, 2017, pp. 1117–1132.
- [8] Md.Mahadi Hasan Nahid and Davood Rafiei, “NormTab: Improving Symbolic Reasoning in LLMs Through Tabular Data Normalization,” arXiv:2406.17961v2 [cs.CL] 2 Apr 2025.
- [9] Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Su, “TableLlama: Towards Open Large Generalist Models for Tables,” arXiv:2311.09206v3 [cs.CL] 4 Apr 2024.
- [10] Arash Dargahi Nobari and Davood Rafiei, “DTT: An Example-Driven Tabular Transformer for Joinability by Leveraging Large Language Models,” arXiv:2303.06748v2 [cs.DB] 25 Dec 2023.
- [11] Arash Dargahi Nobari and Davood Rafiei, “Efficiently Transforming Tables for Joinability,” arXiv:2111.09912v2 [cs.DB] 6 Mar 2022.
- [12] Arash Dargahi Nobari and Davood Rafiei, “TabulaX: Leveraging Large Language Models for Multi-Class Table Transformations,” arXiv:2411.17110v1 [cs.DB] 26 Nov 2024.
- [13] Z. Abedjan et al., “DataXFormer: A Robust Transformation Discovery System,” *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pp. 1134–1145, 2016.
- [14] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed, “Mistral 7B,” arXiv:2310.06825 [cs.CL], 2023.
- [15] A. Grattafiori et al., “The Llama 3 Herd of Models,” arXiv:2407.21783 [cs.AI], 2024.
- [16] M. Mishra et al., “Granite Code Models: A Family of Open Foundation Models for Code Intelligence,” arXiv:2405.04324 [cs.AI], 2024.
- [17] M. Abdin et al., “Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone,” arXiv:2404.14219 [cs.CL], 2024.
- [18] DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” arXiv:2501.12948 [cs.CL], 2025.